

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №5

Выполнил:

Малахов Алексей

БР1.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Подключить и настроить RabbitMQ.
2. Реализовать межсервисное взаимодействие посредством RabbitMQ.

Ход работы

Реализована цепочка асинхронного взаимодействия трёх микросервисов через две очереди RabbitMQ:

```
rental-service → [rental_events] → property-service → [property_events]
               → review-service
```

RabbitMQ добавлен в `docker-compose.yml` (образ `rabbitmq:3-management`, AMQP-порт 5672, веб-интерфейс 15672). Во все три сервиса добавлена зависимость `amqp-lib` и переменная `RABBITMQ_URL`. Все очереди объявлены с флагом `durable: true`, сообщения — с `persistent: true`. При потере соединения сервисы переподключаются автоматически через 5 секунд.

Шаг 1 — Rental Service (publisher). При смене статуса аренды публикует событие в очередь `rental_events`:

```
{ "event": "rental.status_changed", "rental_id": 42,
  "property_id": 1, "property_status": "rented" }
```

`property_status` принимает значения `"rented"` (активация) или `"active"` (завершение / отмена).

Шаг 2 — Property Service (consumer + publisher). Подписывается на `rental_events`, обновляет статус объекта в своей БД, затем публикует событие в очередь `property_events`:

```
{ "event": "property.status_changed", "property_id": 1,
  "property_status": "rented", "rental_id": 42, "owner_id": 5 }
```

Шаг 3 — Review Service (consumer). Подписывается на `property_events`. Когда объект переходит обратно в статус `"active"` (аренда завершена), фиксирует, что отзыв об арендодателе теперь доступен для написания.

Файл	Изменение
<code>rental-service/src/messaging/publisher.ts</code>	Publisher: публикация в <code>rental_events</code>
<code>rental-service/src/controllers/rental.controller.ts</code>	HTTP-вызов на <code>publishRentalStatusChanged()</code> заменён
<code>property-service/src/messaging/consumer.ts</code>	Consumer <code>rental_events</code> + вызов <code>publisher</code>
<code>property-service/src/messaging/publisher.ts</code>	Publisher: публикация в <code>property_events</code>
<code>review-service/src/messaging/consumer.ts</code>	Новый: consumer <code>property_events</code>

Файл	Изменение
docker-compose.yml	rabbitmq:3-management, RABBITMQ_URL в трёх сервисах

Вывод

Реализована цепочка асинхронного межсервисного взаимодействия через RabbitMQ по аналогии с паттерном Producer–Consumer. Rental Service публикует событие смены статуса аренды, Property Service обрабатывает его и передаёт дальше, Review Service получает уведомление о готовности к написанию отзыва. Синхронная зависимость между сервисами устранена: недоступность любого получателя не блокирует работу отправителя.